FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

FUTURE ENHANCEMENTS

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	FUTURE ENHANCEMENTS
Version	v1.0.0
Date	02 July 2026

Potential Features for Next Development Phase

1. OCR — Photograph Prescription

What it is

User photographs their doctor's prescription with their phone camera. The system extracts medicine names and searches automatically.

Implementation Idea

```
Frontend: FileInput → Camera / Gallery picker
          → Send image to backend
Backend:  FastAPI + Tesseract OCR or Google Vision API
          → Extract text from image
          → Parse medicine names from text
          → Return medicine list to frontend
Frontend: Auto-populate search results
```

Frontend Work Needed

- Camera access UI component (`<input type="file" accept="image/*" capture="environment">`)
- Image preview before upload
- Processing state (Skeleton loader)
- `ocrService.js` — POST `/api/v1/ocr/prescription`

2. Voice Search

What it is

User speaks a medicine name. The system converts speech to text and searches.

Implementation Idea

```
Frontend: Button → Web Speech API (SpeechRecognition)
          → Convert speech to text
          → Pass to existing search flow
```

Frontend Work Needed

- `useVoiceSearch` custom hook using `window.SpeechRecognition`
- Microphone button on SearchBar
- Recording state indicator (animated microphone icon)
- No backend changes needed (uses existing search API)

```
function useVoiceSearch() {
  const [transcript, setTranscript] = useState('')
  const [isListening, setIsListening] = useState(false)

  const startListening = () => {
```

```
    const recognition = new window.SpeechRecognition()
    recognition.lang = 'en-IN'
    recognition.onresult = (e) => setTranscript(e.results[0][0].transcript)
    recognition.start()
    setIsListening(true)
  }

  return { transcript, isListening, startListening }
}
```

---

3. Barcode Scanner

What it is

User scans the barcode on a medicine box. The system looks up the medicine instantly.

Implementation Idea

```
Frontend: Camera access → BarcodeDetector API (Chrome)
          → Detect barcode → Extract code
Backend:  GET /api/v1/medicines/barcode/{code}
Frontend: Navigate to MedicineDetailsPage
```

Frontend Work Needed

- `useBarcodeScanner` hook using `window.BarcodeDetector`
- Camera preview modal
- `medicineService.getByBarcode(code)`

---

4. AI-Powered Recommendation

What it is

Instead of simple composition matching, an ML model recommends the best generic based on patient history, location, and effectiveness data.

Implementation Idea

```
Backend:  Train ML model on medicine composition + effectiveness data
          POST /api/v1/recommend { medicineId, userId, location }
          Returns ranked generic list with confidence scores
Frontend: GenericRecommendationPage shows "AI Recommended" badge
          on top-ranked alternatives
```

Frontend Work Needed

- `Badge variant="ai"` — new AI Recommended badge
- Confidence score display in SearchResultCard
- `recommendationService.js`

---

5. Medicine Reminders

What it is

Users set reminders for their medication schedule. The system sends push notifications at the right time.

Implementation Idea

```
Frontend: ReminderPage — add medicine + time + repeat schedule
          Service Worker registration for push notifications
Backend:  Cron job → POST notifications at scheduled times
          Web Push API for browser notifications
```

Frontend Work Needed

- `pages/reminders/RemindersPage.jsx`
- `reminderService.js`
- `useNotificationPermission` hook (request browser push permission)
- Route: `/reminders` in `AppRouter.jsx`

---

6. Delivery Tracking

What it is

Users order medicines online from Jan Aushadhi stores and track delivery.

Implementation Idea

```
Frontend: "Order Now" button on PharmacyCard
          OrderConfirmationPage
          OrderTrackingPage with delivery status timeline
Backend:  Order management API
          Integration with delivery partner API (Dunzo, Swiggy Instamart)
```

Frontend Work Needed

- `pages/orders/` — new page group
- `orderService.js`
- Order status timeline component

---

7. Admin Analytics Dashboard

What it is

Visual charts showing system usage — top searched medicines, user growth, savings achieved.

Implementation Idea

```
Charts library: Recharts or Chart.js
Backend: GET /api/v1/analytics/daily-stats
         GET /api/v1/analytics/top-medicines
         GET /api/v1/analytics/savings-summary
```

Frontend Work Needed

- Integrate `recharts` or `@nivo/core`
- `AdminAnalytics` page expanded with real charts (currently placeholder)
- `analyticsService.js`

---

8. Predictive Inventory

What it is

Admin/pharmacist sees a forecast of which medicines will run out of stock based on historical consumption.

Implementation Idea

```
Backend: ML model trained on inventory depletion data
         GET /api/v1/inventory/forecast
Frontend: InventoryPage adds "Low Stock Predicted" badges
          Admin dashboard shows reorder suggestions
```

---

9. Doctor Module

What it is

Doctors can write digital prescriptions, which are instantly sent to the patient's account.

Implementation Idea

```
New Role: DOCTOR (already in USER_ROLES)
Frontend: DoctorLayout + DoctorDashboard (already has doctor role in routes)
         PrescriptionForm — add medicines + dosage + instructions
         Patient receives prescription as notification
```

Frontend Work Needed

- `pages/doctor/` — new page group
- `DoctorLayout` or reuse `UserLayout` with different nav
- `prescriptionService.js`

---

10. Progressive Web App (PWA)

What it is

The web app becomes installable on Android/iOS and works offline.

Implementation Idea

```
Add vite-plugin-pwa to vite.config.js
Configure: manifest.json (name, icons, theme_color)
           Service Worker (cache assets offline)
           Background sync (queue API calls when offline)
```

Frontend Work Needed

- `vite.config.js` — add `@vite-pwa/vite-plugin`
- `public/manifest.json`
- `public/icons/` — app icons at multiple sizes
- Offline page already built (`OfflinePage.jsx`) — just needs SW integration

```
// vite.config.js addition
import { VitePWA } from 'vite-plugin-pwa'

VitePWA({
  registerType: 'autoUpdate',
  manifest: {
    name: 'Smart Medicine System',
    short_name: 'SmartMed',
    theme_color: '#2563eb',
    icons: [{ src: '/icon-192.png', sizes: '192x192' }]
  }
})
```

---

11. Chatbot Support

What it is

AI chatbot answers medicine-related questions ("What is the generic for Crocin?", "Are there any Jan Aushadhi stores near me?").

Implementation Idea

```
Frontend: Floating chat widget (bottom-right corner)
          ChatMessage component
          Typing indicator (...)
Backend:  LLM API (OpenAI GPT or open-source LLM)
          POST /api/v1/chat { message, userId }
          Returns { reply }
```

Frontend Work Needed

- `ChatWidget.jsx` — floating chat bubble
- `useChatSession` hook
- `chatService.js`

---

12. Multilingual Support

What it is

UI available in Hindi, Marathi, Tamil, Telugu, and other Indian languages.

Implementation Idea

```
Library: react-i18next
Frontend: Add i18n configuration
           Translation JSON files per language
           Language selector in TopBar
```

Since this is a Janaushadhi app targeting rural India, Hindi support is particularly valuable.

Priority for Next Phase

Enhancement	Effort	Impact
PWA (installable)	Low	High — mobile users
Voice Search	Low	High — literacy accessibility
Medicine Reminders	Medium	High — adherence
OCR Prescription	Medium	High — core feature
Multilingual	Medium	Very High — reach
AI Recommendation	High	High — differentiation
Barcode Scanner	Low	Medium
Delivery Tracking	High	Medium